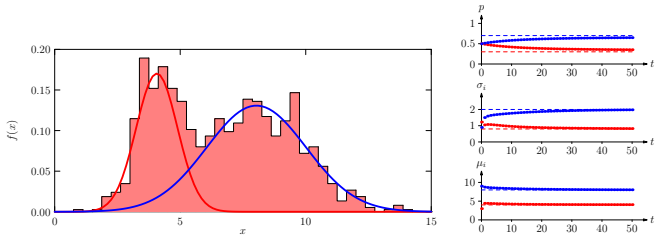
Mathematics of Machine Learning

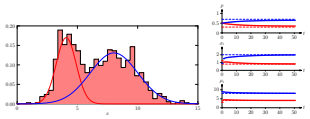
Probabilistic Inference



Generative Models, Hierarchical Models, MAP, Empirical Bayes, Mixture of Gaussians, Expectation Maximisation

Outline

1. **Building Probabilistic Models**
2. Hierarchical Models
3. Solution Strategies
4. Mixtures and EM



Beyond Conjugate Priors

- In the Bayes lecture we chose priors so that the posterior had the same form (conjugate priors)
- Real problems have many interacting random variables, some of which we never observe, and the posterior has no closed form
- This lecture: how to set up a probabilistic model, and simple strategies for using it (MAP, empirical Bayes, EM)
- The MCMC lecture shows how to compute the full posterior

Two Rules

- Sum rule (marginalisation; see the probability lecture)

$$\mathbb{P}(X) = \sum_Y \mathbb{P}(X,Y) \qquad f(x) = \int f(x,y)\mathrm{d}y$$

- Product rule (the definition of conditional probability)

$$\mathbb{P}(X,Y) = \mathbb{P}(X|Y)\mathbb{P}(Y) = \mathbb{P}(Y|X)\mathbb{P}(X)$$

- Bayes' rule is the product rule rearranged: $\mathbb{P}(X|Y) = \mathbb{P}(Y|X)\mathbb{P}(X)/\mathbb{P}(Y)$
- Everything else follows from these two rules

Discriminative and Generative Models

- A **discriminative model** gives the prediction directly, e.g. $\mathbb{P}(C|\boldsymbol{x})$ for a class C given features $\boldsymbol{x}$ (as for the Bayes' optimal discrimination surface in *foundations of machine learning*)
- A **generative model** describes how the features and the output are generated together: $\mathbb{P}(\boldsymbol{X},Y)$
- Generative models are often more natural to think about
- And we can still use them to discriminate

$$\mathbb{P}(Y|\boldsymbol{X}) = \frac{\mathbb{P}(\boldsymbol{X},Y)}{\mathbb{P}(\boldsymbol{X})} = \frac{\mathbb{P}(\boldsymbol{X},Y)}{\sum_Y \mathbb{P}(\boldsymbol{X},Y)}$$

Latent Variables

- Sometimes we have models that involve random variables that we don't observe and we don't care about
- These are called **latent variables**
- They are not observed and not our final target, but they make the model easy to write down
- If we have a latent variable Z and observed variable $\boldsymbol{X}$ and we are predicting a variable Y then we would **marginalise** over the latent variable

$$\mathbb{P}(\boldsymbol{X},Y) = \sum_Z \mathbb{P}(\boldsymbol{X},Y,Z)$$

Generative Stories

- Build a model by writing down how the data could have been generated (a simulator), each line drawing one variable given earlier ones

```
theta = rng.beta(a, b, size=J)     # a rate for each product
k = rng.binomial(n, theta)         # observed positive reviews
```

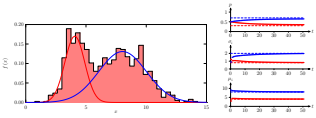
- The joint distribution is the product of the lines (product rule)

$$f(\boldsymbol{\theta},\boldsymbol{k}) = \prod_{j=1}^{J} \mathrm{Beta}(\theta_j|a,b)\,\mathrm{Bin}(k_j|n_j,\theta_j)$$

- **Probabilistic programming languages** (e.g. Stan, PyMC) take such a programme and do the inference for you

Outline

1. Building Probabilistic Models
2. **Hierarchical Models**
3. Solution Strategies
4. Mixtures and EM



Rating Products

- A shop lists products with the fraction of positive reviews: which is better?

	positive	fraction
Product A	3 of 3	100%
Product B	1840 of 2000	92%

- Ranking by the fraction puts every product with a handful of good reviews at the top
- Intuitively we should trust A's 100% less, and something like "most products are rated about 80%" should pull it down: a hierarchical model makes this precise

A Model for Ratings

- Product j has an unknown probability θ_j that a review is positive; k_j of its n_j reviews are positive

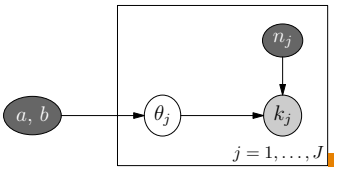
$$k_j \sim \mathrm{Bin}(n_j, \theta_j)$$

- The products are different but similar: their θ_j come from a common population

$$\theta_j \sim \mathrm{Beta}(a, b)$$

- The **hyperparameters** a, b describe the population: $a/(a+b)$ is the typical rate and $a+b$ measures how alike the products are
- The model has levels: population → product → reviews (a **hierarchical model**)

Graphical Models



- Nodes are variables (dark: fixed; shaded: observed; white: latent); arrows point from the variables each one is drawn from
- A box (a **plate**) means "repeat for each j"
- The joint distribution is the product over nodes of each node given its parents
- Diagrams like this are a good way to design and explain models

One Product at a Time

- If we knew a and b, each product is a beta-binomial problem from the Bayes lecture

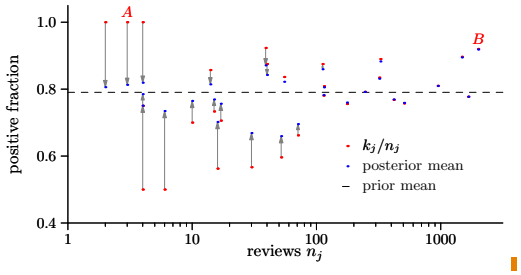
$$f(\theta_j|k_j) = \mathrm{Beta}(\theta_j|a+k_j, b+n_j-k_j)$$

- The posterior mean is a weighted average: the prior counts as $a+b$ extra reviews

$$\mathbb{E}[\theta_j|k_j] = \frac{a+k_j}{a+b+n_j} = \frac{a+b}{a+b+n_j}\frac{a}{a+b} + \frac{n_j}{a+b+n_j}\frac{k_j}{n_j}$$

- With $a=20$, $b=5$ (typical rate 0.8, worth 25 reviews): A gets $23/28 = 0.82$ and B gets $1860/2025 = 0.92$: B is better

Shrinkage



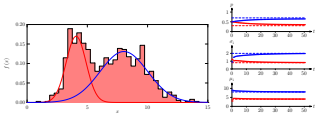
Few reviews: pulled (**shrunk**) to the typical rate; many: barely move. Products **borrow strength** from each other.

Partial Pooling

- Two extremes
  - ⋆ no pooling: estimate each product alone (k_j/n_j)—noisy when n_j is small
  - ⋆ complete pooling: assume all products are the same—ignores real differences
- The hierarchical model sits in between, and the data decide where (through $a+b$)
- The same idea appears everywhere: schools, hospitals, adverts, players, users of a recommender
- But we don't know a and b, and with them unknown the posterior $f(a,b,\boldsymbol{\theta}|\boldsymbol{k})$ has no closed form

Outline

1. Building Probabilistic Models
2. Hierarchical Models
3. **Solution Strategies**
4. Mixtures and EM



Full Bayesian Inference

- Bayes' rule gives the posterior over the parameters

$$\mathbb{P}(\boldsymbol{\Theta}|\mathcal{D}) = \frac{\mathbb{P}(\mathcal{D}|\boldsymbol{\Theta})\mathbb{P}(\boldsymbol{\Theta})}{\mathbb{P}(\mathcal{D})}$$

- It describes its own uncertainty and is much less prone to over-fitting (we average over parameters rather than choosing the best)
- But the posterior is rarely a nice function, and its normalisation, the *evidence* $\mathbb{P}(\mathcal{D}) = \sum_{\boldsymbol{\Theta}} \mathbb{P}(\mathcal{D}|\boldsymbol{\Theta})\mathbb{P}(\boldsymbol{\Theta})$, is a sum over far too many values
- Monte Carlo methods (the MCMC lecture) can sample from the posterior without computing $\mathbb{P}(\mathcal{D})$

Maximum A Posteriori (MAP) Solution

- Work around: the mode of the posterior (no need for $f(\mathcal{D})$)

$$\boldsymbol{\Theta}_{\text{MAP}} = \operatorname*{argmax}_{\boldsymbol{\Theta}} f(\mathcal{D}|\boldsymbol{\Theta}) f(\boldsymbol{\Theta}) = \operatorname*{argmax}_{\boldsymbol{\Theta}} \log(f(\mathcal{D}|\boldsymbol{\Theta})) + \log(f(\boldsymbol{\Theta}))$$

- $-\log(f(\boldsymbol{\Theta}))$ acts as a regulariser: a Gaussian prior on weights gives $\|\boldsymbol{w}\|^2$, i.e. ridge regression or weight decay (we use this in the next lecture)
- With a uniform prior MAP is **maximum likelihood**: often sufficient for *government work*
- Neither is Bayesian (despite what you are sometimes told): they can overfit and give no error estimate

Empirical Bayes

- A half-way house: integrate out the many θ_j, optimise the few hyperparameters, then use the conjugate posteriors
- For the ratings model the integral is in closed form (the **beta-binomial**)

$$\begin{aligned}\mathbb{P}(k_j|a,b) &= \int_0^1 \text{Bin}(k_j|n_j,\theta)\,\text{Beta}(\theta|a,b)\,\mathrm{d}\theta \\ &= \binom{n_j}{k_j}\frac{B(a+k_j,b+n_j-k_j)}{B(a,b)}\end{aligned}$$

- Choose a, b to maximise the evidence $\prod_j \mathbb{P}(k_j|a,b)$ (as for the GP hyperparameters): for the data in the figure $a = 19.9$, $b = 5.3$
- Cheap and often good, but ignores uncertainty in a, b

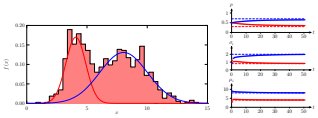
A Ladder of Strategies

Strategy	What you get	Cost
ML	a point estimate	optimiser
MAP	a regularised point	optimiser
Empirical Bayes	posteriors given a,b	an integral
EM	ML/MAP with latents	iterate
Full Bayes	all the uncertainty	MCMC

- Choose the cheapest that answers your question
- The rest of this lecture: EM

Outline

1. Building Probabilistic Models
2. Hierarchical Models
3. Solution Strategies
4. **Mixtures and EM**



You Already Know EM

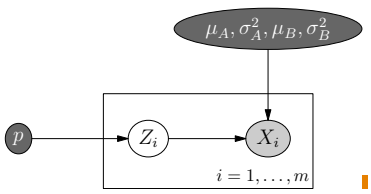
- k-means clustering
  1. assign each point to its nearest centre
  2. move each centre to the mean of its points
  3. repeat
- Step 1 guesses the hidden cluster labels; step 2 fits the parameters as if those guesses were right
- The EM algorithm does the same with **probabilities** of belonging to each cluster
- It comes from a probabilistic model: a mixture

Mixture of Gaussians

- Suppose we detect two types of short-lived particle, A or B, and measure each particle's mass, X_i, with a noisy detector, but not its type
- Let $Z_i \in \{0,1\}$ indicate that particle i is of type A, with $\mathbb{P}(Z_i = 1) = p$
- X_i is normal with unknown means and variances, so the parameters are $\boldsymbol{\Theta} = (\mu_A,\sigma_A^2,\mu_B,\sigma_B^2,p)$
- The probability of X_i is given by

$$f(X_i|Z_i,\boldsymbol{\Theta}) = Z_i\mathcal{N}\big(X_i\big|\mu_A,\sigma_A^2\big) + (1-Z_i)\mathcal{N}\big(X_i\big|\mu_B,\sigma_B^2\big)$$

Mixture as a Graphical Model

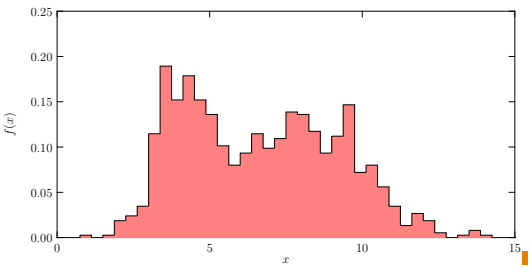


- For each data point draw a type Z_i, then a mass X_i from that type's normal distribution
- Z_i is latent: we must marginalise it out
- The parameters (dark) are shared by all data points, so they sit outside the plate

Data

- Note that

$$\begin{aligned}f(X_i|\boldsymbol{\Theta}) &= \sum_{Z_i\in\{0,1\}} f(X_i,Z_i|\boldsymbol{\Theta}) = \sum_{Z_i\in\{0,1\}} f(X_i|Z_i,\boldsymbol{\Theta})\mathbb{P}(Z_i) \\ &= \mathbb{E}_{Z_i}[f(X_i|Z_i,\boldsymbol{\Theta})] = p\mathcal{N}\big(X_i\big|\mu_A,\sigma_A^2\big) + (1-p)\mathcal{N}\big(X_i\big|\mu_B,\sigma_B^2\big)\end{aligned}$$



Maximum Likelihood

- A full Bayesian treatment needs priors on $\boldsymbol{\Theta}$ and would give a distribution over parameters
- Often we only want a reasonable estimate for some of our parameters (e.g. the masses μ_A and μ_B)
- A reasonable approach is to seek those parameters that maximise the likelihood of our observed data (assumed independent)

$$f(\mathcal{D}|\boldsymbol{\Theta}) = \prod_{X \in \mathcal{D}} f(X|\boldsymbol{\Theta})$$

- Strictly, we seek a good **local** maximum: putting $\mu_A = X_1$ and letting $\sigma_A \to 0$ makes the likelihood infinite

EM Algorithm

- If we knew each particle's type, Z_i, maximising the likelihood would be easy, but estimating $\mathbb{P}(Z_i = 1|X_i)$ needs the parameters
- The **expectation-maximisation** (EM) algorithm iterates: maximise the log-likelihood of the data **and** latent variables, averaged over the latent variables using the current parameters

$$\boldsymbol{\Theta}^{(t+1)} = \operatorname*{argmax}_{\boldsymbol{\Theta}} \sum_{\boldsymbol{Z}} \mathbb{P}\Big(\boldsymbol{Z}\Big|\mathcal{D},\boldsymbol{\Theta}^{(t)}\Big) \log\big(f(\mathcal{D},\boldsymbol{Z}|\boldsymbol{\Theta})\big)$$

- E-step: compute $\mathbb{P}\big(\boldsymbol{Z}|\mathcal{D},\boldsymbol{\Theta}^{(t)}\big)$; M-step: maximise
- No step decreases the likelihood (the proof is in the notes)

E-Step: Responsibilities

- We need the distribution of the latent variables conditioned on the data and current estimated parameters
- For our problem

$$\mathbb{P}\Big(\boldsymbol{Z}\Big|\mathcal{D},\boldsymbol{\Theta}^{(t)}\Big) = \prod_{i=1}^{m} \mathbb{P}\Big(Z_i\Big|X_i,\boldsymbol{\Theta}^{(t)}\Big)$$

where, by Bayes' rule, the **responsibility** of type A for point i is

$$\gamma_i^{(t)} = \mathbb{P}\Big(Z_i = 1\Big|X_i,\boldsymbol{\Theta}^{(t)}\Big) = \frac{p^{(t)}\mathcal{N}\Big(X_i|\mu_A^{(t)},\sigma_A^{2(t)}\Big)}{p^{(t)}\mathcal{N}\Big(X_i|\mu_A^{(t)},\sigma_A^{2(t)}\Big) + (1-p^{(t)})\mathcal{N}\Big(X_i|\mu_B^{(t)},\sigma_B^{2(t)}\Big)}$$

and $\mathbb{P}\big(Z_i = 0\big|X_i,\boldsymbol{\Theta}^{(t)}\big) = 1 - \gamma_i^{(t)}$

M-Step

- The data points are independent, so

$$Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)}) = \sum_{i=1}^{m}\Big[\gamma_i^{(t)}\Big(\log(p) - \frac{(X_i-\mu_A)^2}{2\sigma_A^2} - \tfrac{1}{2}\log\big(2\pi\sigma_A^2\big)\Big) + (1-\gamma_i^{(t)})\Big(\log(1-p) - \frac{(X_i-\mu_B)^2}{2\sigma_B^2} - \tfrac{1}{2}\log\big(2\pi\sigma_B^2\big)\Big)\Big]$$

- For example, only the first bracket involves μ_A

$$\frac{\partial Q}{\partial \mu_A} = \sum_{i=1}^{m} \gamma_i^{(t)} \frac{X_i - \mu_A}{\sigma_A^2} = 0 \quad \Rightarrow \quad \mu_A^{(t+1)} = \frac{\sum_{i=1}^m \gamma_i^{(t)} X_i}{\sum_{i=1}^m \gamma_i^{(t)}}$$

- The other parameters work the same way (see the notes)

Update Equations

- Fraction of type A: the average responsibility

$$p^{(t+1)} = \frac{1}{m}\sum_{i=1}^{m} \gamma_i^{(t)}$$

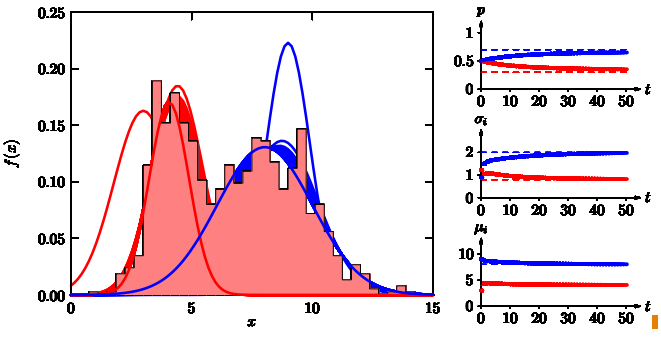
- Means: weighted averages of the data

$$\mu_A^{(t+1)} = \frac{\sum_{i=1}^m \gamma_i^{(t)} X_i}{\sum_{i=1}^m \gamma_i^{(t)}}, \qquad \mu_B^{(t+1)} = \frac{\sum_{i=1}^m (1-\gamma_i^{(t)}) X_i}{\sum_{i=1}^m (1-\gamma_i^{(t)})}$$

- Variances: weighted variances (if every γ_i were 0 or 1, these would all be the ordinary group statistics)

$$\sigma_A^{2(t+1)} = \frac{\sum_{i=1}^m \gamma_i^{(t)} (X_i - \mu_A^{(t+1)})^2}{\sum_{i=1}^m \gamma_i^{(t)}}$$

Example



EM in Practice

- With equal, fixed variances and $\sigma \to 0$ the responsibilities become 0 or 1: EM becomes k-means
- EM finds a local maximum: run it from several starts
- A component can collapse onto a single point ($\sigma \to 0$, likelihood $\to \infty$) a prior on σ (i.e. MAP) prevents this
- Convergence can be slow when the components overlap (the latent variables are very uncertain)
- Simple updates, no step size, and it works for any model with latent variables (missing data, hidden Markov models, . . .)

Summary

- Build a model by writing down how the data were generated: a generative story or graphical model
- Hierarchical models let similar groups share strength (shrinkage)
- Beyond conjugate priors there is a ladder of strategies: ML, MAP, empirical Bayes, EM, full Bayes with MCMC
- The EM algorithm maximises likelihoods (or MAP) when there are latent variables
- It gives nice update equations, but finds local optima and can converge slowly